vaadin create /26

Barcelona, Spain

27 October 2026



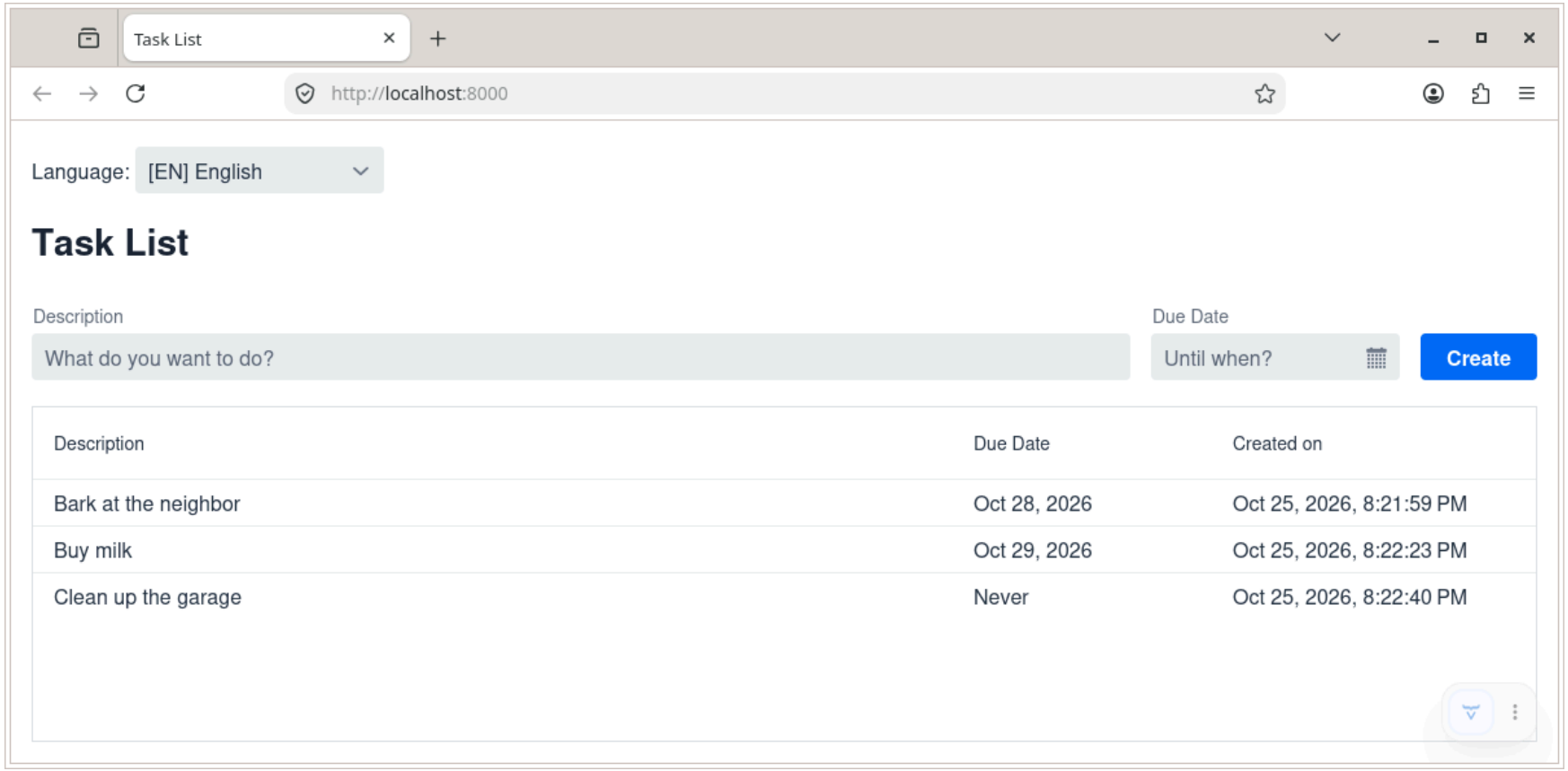
vaadin



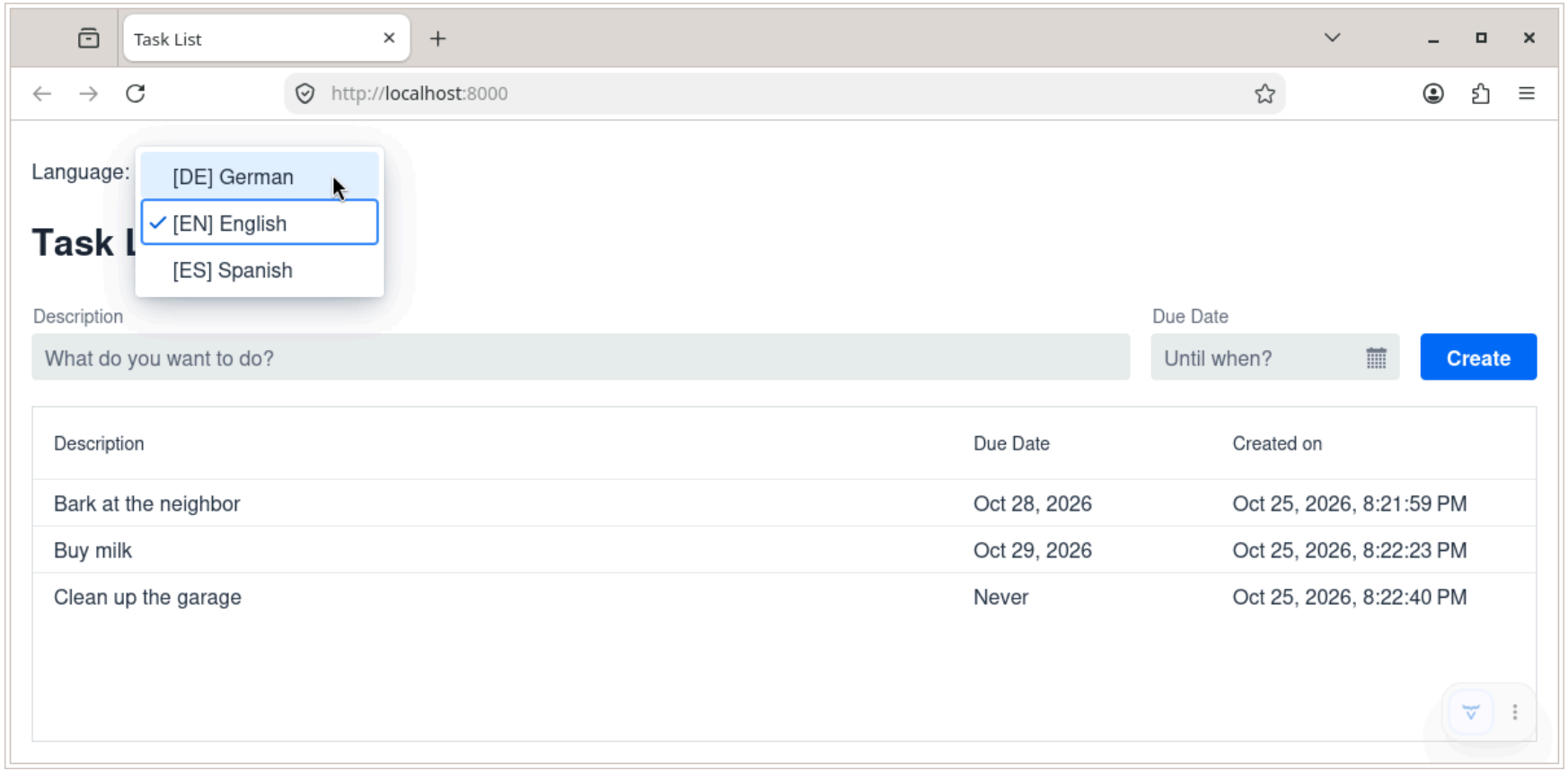
Rüdiger Schobbert

A Type Safe I18N Solution with Vaadin Flow

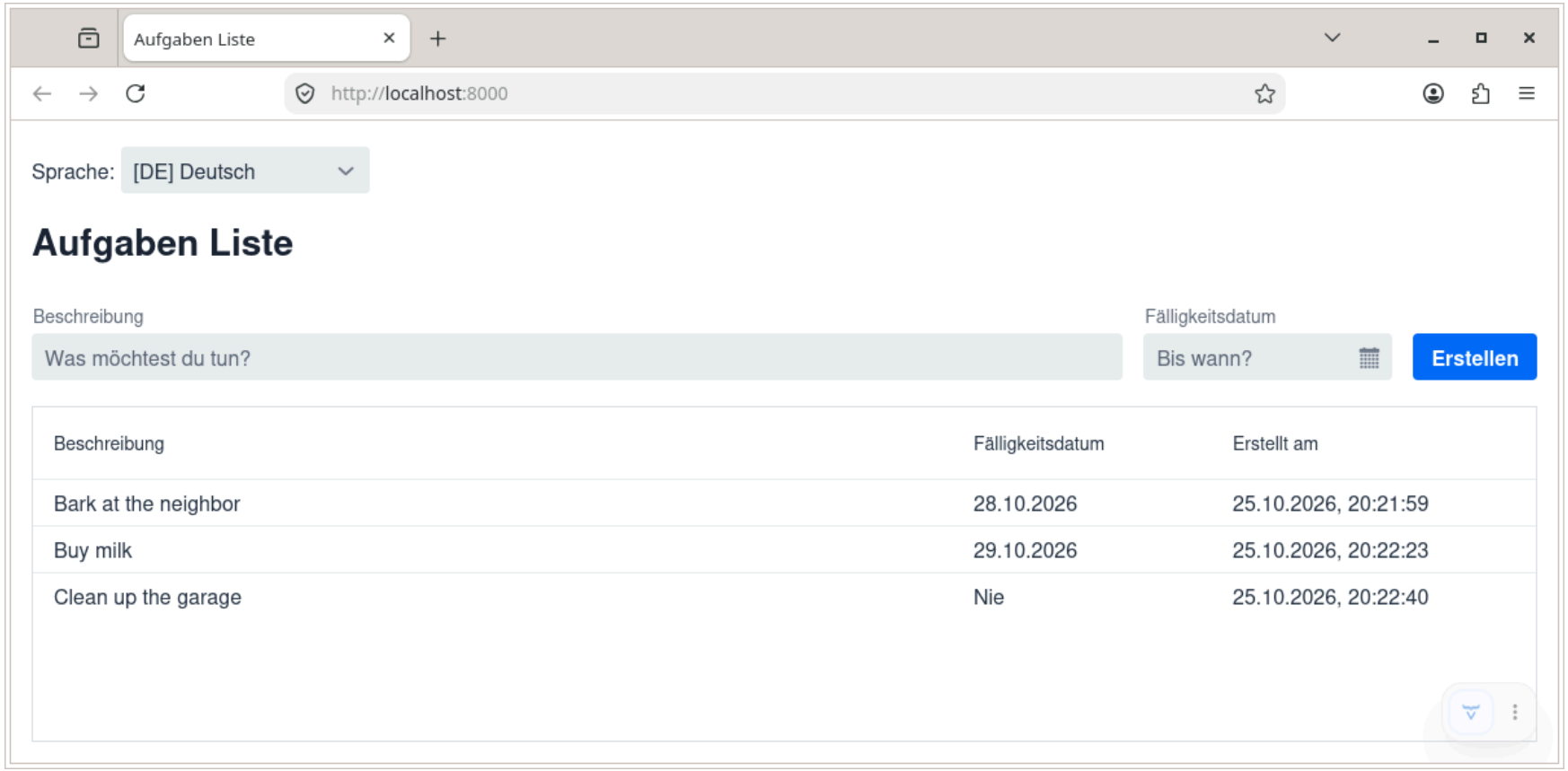
Demo Project: A very simple Tasklist Application



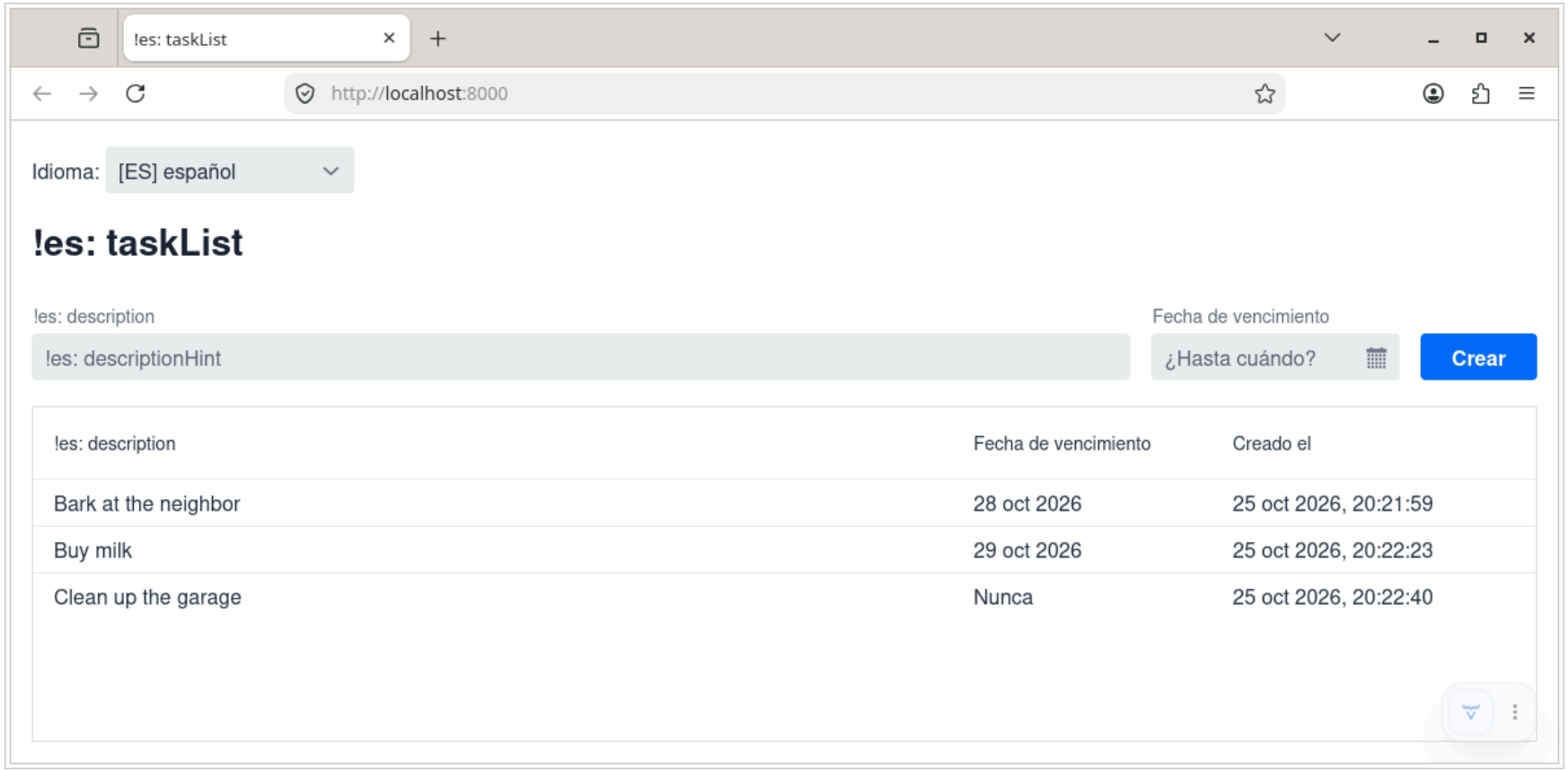
Demo Project: About to select German



Demo Project: The Tasklist Application in German



Demo Project: The Problem in Spanish



Agenda

- I18N the classic way
- My solution outlined
- History / Why am I here
- My solution in Detail
- Improvements
- Quick Summary
- Questions / Discussion



Classic I18N: Translations in multiple Files

src/main/resources/vaadin-i18n/translations_de.properties

```
language=Sprache
taskList=Aufgaben Liste
```

src/main/resources/vaadin-i18n/translations_en.properties

```
language=Language
taskList=Task List
```

Put `translationKey=value` in translation files:

- in **src/main/resources/vaadin-i18n/** if you use Vaadin's `DefaultI18NProvider`
  - `translations.properties`: default translations (fallback)
  - `translations_LANGUAGE_COUNTRY_VARIANT.properties`
    - `translations_de.properties`
    - `translations_en.properties`
    - ...



Classic I18N: how GUI Components get Translations

```
public class TaskListView extends VerticalLayout {

    H2 viewHeading = new H2();  // <h2> view heading text </h2>

    public TaskListView() {
        // getTranslation(...)  from  com.vaadin.flow.component.Component
        viewHeading.setText( getTranslation("taskList") );

        // parameters are not type checked
        String errorMsg1 = getTranslation("my.err.msg", 1);
        String errorMsg2 = getTranslation("my.err.msg", "a");
    }
}
```



Classic I18N: Vaadin **Component** Class

```
package com.vaadin.flow.component;

public abstract class Component implements ... {

    public String getTranslation(String key, Object... params) {
        // I18NProvider i18nProvider = ... current I18NProvider or null
        // Locale       locale       = ... evaluate the correct Locale for user / browser

        // return translation from i18nProvider  or  "!{" + key + "}!"
    }

    // ... more getTranslation(...) methods

}
```



Classic I18N: Vaadin **I18NProvider** Interface

```java
package com.vaadin.flow.i18n;

public interface I18NProvider extends Serializable {

    List<Locale> getProvidedLocales();

    String getTranslation(String key, Locale locale, Object... params);

    // ... some more ...
}
```



Classic I18N: Vaadin **DefaultI18NProvider** Class

```java
package com.vaadin.flow.i18n;
public class DefaultI18NProvider implements I18NProvider {

    public DefaultI18NProvider(List<Locale> providedLocales, ...) { ... }

    @Override
    public List<Locale> getProvidedLocales() { return providedLocales; }

    @Override
    public String getTranslation(String key, Locale locale, Object... params) {
        if (key == null) { return ""; }
        ResourceBundle bundle = getBundle(locale);
        if (bundle == null) { return key; }
        try {
            String value = bundle.getString(key);
            return (params.length > 0)
                ? new MessageFormat(value, locale).format(params)
                : value;
        } catch (final MissingResourceException e) {
            return "!" + locale.getLanguage() + ": " + key;
        }
    }
}
```



Classic I18N: **Summary**

- **Standard Vaadin Apps** usually solve I18N via `I18NProvider`
  - with **String translation keys**
  - and **Object parameters**
- Use `DefaultI18NProvider` / write your own by implementing `I18NProvider`
- Use `getTranslation(...)` in your GUI Components
  - `I18NProvider` will then lookup translation in ResourceBundle (translation_XX.properties)



Motivation: Classic I18n **Pain Points**

- **No native IDE support:**
  - No autocomplete or navigation (unless special IDE support)
- **No compile-time verification:**
  - Typos in string keys (e.g. `"tasklist"`) are (silently) visible at runtime: `"!es: tasklist"`
  - usually no Exception is thrown
- **Fragile parameter handling:**
  - Missing argument: `"welcome {0}"` → `"welcome "` instead of `"welcome Joe"`
  - Wrong arguments: `"size is {0,number,#.##} MB"` → `getTranslation("fileSize", "NaN")`

    ⇒ `IllegalArgumentException: Cannot format given Object as a Number`
- **Refactoring nightmare:**
  - Renaming or deleting keys in `.properties` files, **breaks** you application **silently**



Inspiration: I18N in GWT (Google Web Toolkit)

Years ago, GWT introduced a type-safe I18N approach using Java interfaces

```java
import com.google.gwt.i18n.client.Constants;

public interface AppLabels extends Constants {

    @DefaultStringValue("Language")
    String language();

    @DefaultStringValue("Task List")
    String taskList();
}
```

```java
import com.google.gwt.i18n.client.Messages;

public interface AppMessages extends Messages {

    @DefaultMessage("Field '{0}' is required")
    String fieldRequired(String fieldName);
}
```



Inspiration: GWT (Google Web Toolkit)

Why we wanted this back:

- **IDE Auto-Completion:**
  - **Available translation keys** (methods) show up via Code Completion / IntelliSense
- **Strict Type Safety:**
  - Method signatures enforce **correct argument types** (e.g., `String`, `int`) at compile time.
- **Seamless Refactoring:**
  - **Renaming a method** updates all occurrences instantly across the workspace.



Realization: Java Dynamic Proxy

```java
public interface I18n {
    void switchLanguage(Language language);

    @TranslationDE("Sprache")
    @TranslationEN("Language")
    String language();

    @TranslationDE("Aufgaben Liste")
    @TranslationEN("Task List")
    String taskList();

    @TranslationDE("Das Feld '{0}' darf nicht leer sein!")
    @TranslationEN("Field '{0}' is required")
    String fieldRequired(String fieldName);
}
```

- **100% Type-Safe** without the overhead of compile-time code generation or annotation processors.
- Uses core Java `java.lang.reflect.Proxy` to intercept UI translation calls at runtime.



History

- The framework was developed because
  - Google abandoned GWT (e.g. no new versions for new JDKs)
  - We had an important customer with a GWT frontend
  - I decided to switch to Vaadin

    (after hoping for some time that GWT would revive)
  - String translation keys were no option for me
- It was developed while migrating the customer from GWT to Vaadin
  - That was many years ago
  - **The application/framework has been running in production since then**
- The **Basic Idea** of this framework → **Demo Projects**



Demo Projects

- Show the basic idea (pealed-out) of the original framework
- start easy
- build upon another (demo-00 → demo-40)
- get more and more powerful
- `demo-00-classic-way`
- `demo-10-simple-vaadin14-java8`
- `demo-15-simple-vaadin25-java21` (← start here) Shows the **basic idea** of the framework
- `demo-20-better-vaadin25-java21`
- `demo-30-framework-vaadin25-java21`
- `demo-40-framework-vaadin25-java21`



The @TranslationXX Annotations

- Create one annotation for each **supported** language
  - write it with exactly one method `String value()`
  - must be marked with `RetentionPolicy.RUNTIME`
- e.g. `@TranslationDE` and `@TranslationEN` to support German and English

```java
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.RUNTIME)  // essential: annotations are necessary at runtime
public @interface TranslationDE {
    String value();
}
```

```java
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.RUNTIME)
public @interface TranslationEN {
    String value();
}
```



The I18n interface

- Central place where to define translation keys (as methods) and default translations
- you can use the translation keys for GUI labels and (parameterized) messages
- (it also contains the special `switchLanguage` method)

```
public interface I18n {
    void switchLanguage(Language language);

    @TranslationDE("Sprache")
    @TranslationEN("Language")
    String language();

    @TranslationDE("Aufgaben Liste")
    @TranslationEN("Task List")
    String taskList();

    @TranslationDE("Das Feld '{0}' darf nicht leer sein!")
    @TranslationEN("Field '{0}' is required")
    String fieldRequired(String fieldName);
}
```



I18nBundle is a Proxy Factory

- uses `java.lang.reflect.Proxy.newProxyInstance(...)` to create a new proxy
- has an internal class `I18nHandler` (`implements InvocationHandler`) with a `Language` state

```
public class I18nBundle {

    I18n get(Language language) {
        return (I18n)Proxy.newProxyInstance( i18nClassLoader, proxyInterfaces, new I18nHandler(language) );
    }

    class I18nHandler implements java.lang.reflect.InvocationHandler {   // NON-STATIC class
        private Language language;
        ....
    }
}
```

```
package java.lang.reflect;

public class Proxy implements Serializable {

    public static Object newProxyInstance( ClassLoader cl,  Class<?>[] proxyInterfaces, InvocationHandler h ) { .... }
}
```



I18nHandler is the InvocationHandler of the Proxy

- has a `Language` Property which can be changed with `I18n.switchLanguage(Language)`
- calls `getTranslation(....)` in outer `I18nBundle` (singleton) object

```
public class I18nBundle {
    String getTranslation(Language language, String translationKey, Object[] args) { ... }

    class I18nHandler implements java.lang.reflect.InvocationHandler {
        private Language language;

        private I18nHandler(Language language) {  this.language = language;  }

        @Override
        public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
            if (method.getName().equals("switchLanguage")) {
                this.language = ((Language)args[0]);
                return null;
            } else {
                return getTranslation( this.language, method.getName(), args );
            }
        }
    }
}
```



I18nBundle is also a Translation Cache

- has an internal cache of all translations
- the translation cache is created at startup by `I18nMethodReader` helper class

```
public class I18nBundle {
    final LabelAndMsgCache lblsMsgsCache;  // 'LabelAndMsgCache' is a fake name to describe the intention

    public I18nBundle() {
        lblsMsgsCache = new I18nMethodReader(I18n.class).read();
    }

    String getTranslation(Language language, String translationKey, Object[] args) {
        return lblsMsgsCache.get(language, translationKey, args);
    }


    I18n get(Language language) { return (I18n)Proxy.newProxyInstance( .... ); }

    class I18nHandler implements java.lang.reflect.InvocationHandler { .... }
}
```



I18nMethodReader

`I18nMethodReader` builds the translation cache by scanning the `I18n` interface

```
public class I18nMethodReader {

    I18nMethodReader(Class<? extends I18n> i18nClass) { ... }

    LabelAndMsgCache read() throws IllegalStateException {  // LabelAndMsgCache is a
        // ...                                                 fake name to describe the intention
        for (Method method : i18nClass.getMethods()) {
            if (method.getName().equals("switchLanguage")) {
                // remember switchLanguage method, return it to I18nBundle afterwards
            } else {
                if (lblsMsgs.containsKey(method.getName())) {
                    // throw an Exception or handle as configured
                }
                // get all @TranslationXX annotations of the method
                // and get the actual "value" (= translation)
            }
        }
        consistencyCheck( ... );
        // ... return the created cache ...
    }
}
```



The Language Enum

```
public enum Language { // just for simple demonstration purposes
    DE( /* mandatory= */ true , TranslationDE.class),
    EN( /* mandatory= */ true , TranslationEN.class),

    private final Locale  locale = Locale.forLanguageTag(name().toLowerCase());
    private final boolean mandatory;
    private final Class<? extends Annotation> annotationClass;

    private Language(boolean mandatory, Class<? extends Annotation> annotationClass) {
        this.mandatory       = mandatory;
        this.annotationClass = annotationClass;
    }
    public int     getIndex()    { return ordinal(); }
    public boolean isMandatory() { return mandatory; }
    public Locale  getLocale()   { return locale;    }

    public String getName() { return locale.getDisplayLanguage(locale); }

    public String getName(Language nameLanguage) {
        return locale.getDisplayLanguage( nameLanguage.getLocale() );
    }
}
```



Summary: Core Architectural Concepts

- `I18n` interface(s):
  - Central item(s) for GUI Components, where **translation keys** are **defined as Java methods**
  - Annotate the basic (and mandatory) translations directly to the method (= translationKey)
- `@TranslationXX` annotations:
  - One annotation for each Language
- `I18nMethodReader` Helper class of I18nBundle:
  - Scans the interface(s) **at application startup** and builds the translation cache
    - reads all methods and their `@TranslationXX` annotations
    - runs consistency / completeness checks
- `I18nBundle` (Singleton) Caching Layer:
  - Stores translations in a cache for fast lookup.
  - Creates Dynamic Proxies with a **stateful** InvocationHandler `I18nHandler`
    - `I18nHandler` is the interception layer

      → uses `I18nBundle` to resolve translationKeys **or** reacts to `switchLanguage()`



Service in Spring: I18nBundleService

- Since `I18nBundle` is an internal class, we hide it behind a Spring Service

```
@Service
public class I18nBundleService {
    private final I18nBundle i18nBundle;

    public I18nBundleService(I18nBundle i18nBundle) {
        this.i18nBundle = i18nBundle;
    }

    public I18n getI18n(Language language) {
        return i18nBundle.get(language);
    }
}
```

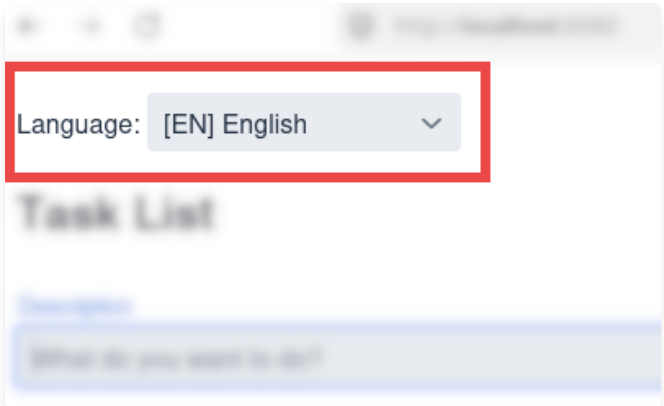


Clean Architecture: Strict GUI Decoupling

- All GUI Components access only
  - `I18n` interface
- `AppLanguageWrapper` additionally
  - uses `I18nBundleService` to get the Proxy
  - is annotated with `@UIScope`
    - each Browser Tab independent of each other
- Alternatives to tight coupling (TaskListView → AppLanguageWrapper):
  - Implement / Fire an event like `myapp.LanguageChangeEvent`



GUI Layer: The Language Select Box (Part 1)



- `AppLanguageWrapper` has four key responsibilities:
  - It's a GUI Component (Label and Select Box)
  - Intercepts user interaction on language select box
  - Shifts language state in `I18nHandler` (via Proxy)
  - Triggers Vaadin's `LocaleChangeEvent` (calls Components implementing `LocaleChangeObserver`)

```
@Component
@UIScope   // UIScope => every UI (browser tab) has its own language state
public class AppLanguageWrapper extends Div {
    Span             label          = new Span();
    Select<Language> languageSelect = new Select<>();
    I18n             i18n;
    public AppLanguageWrapper(I18nBundleService i18nBundleService) {
        i18n = i18nBundleService.getI18n(getInitialLanguage());
        // setItems in select box, register ValueChangeListener, set label text, ...
    }
}
```

GUI Layer: The Language Select Box (Part 2)

```
public class AppLanguageWrapper extends Div {
    Span             label          = new Span();
    Select<Language> languageSelect = new Select<>();
    I18n             i18n;

    // ...

    void onLanguageSelected(ComponentValueChangeEvent<...> event) {
        I18nLanguage newLanguage = event.getValue();
        i18n.switchLanguage(newLanguage);                    // Shift I18nHandler State via Proxy
        UI.getCurrent().setLocale(newLanguage.getLocale()); // Propagate to LocaleChangeObserver
        updateUi18n();                                       // Refresh this wrapper's own texts
    }
    void updateUi18n() {
        languageSelect.getDataProvider().refreshAll(); // Caution: call to setValue necessary afterwards
        label.setText(i18n.language() + ": ");         // Translate the label via our proxy
    }
}
```



GUI Layer: Language Change Notification

- Views (or other GUI components) implement Vaadin's `LocaleChangeObserver`
- They receive a `LocaleChangeEvent` once `AppLanguageWrapper` propagates the change

  → adjust their internationalized texts

```
@Route(value = "")
public class TaskListView  extends     VerticalLayout
                           implements  LocaleChangeObserver
{
    private final I18n i18n;
    private final H2 viewTitleHeader = new H2();

    public TaskListView(TaskService taskService, AppLanguageWrapper appLanguageWrapper) {
        this.i18n = appLanguageWrapper.getI18n();
        // ...
    }

    @Override
    public void localeChange(LocaleChangeEvent event) {
        viewTitleHeader.setText(i18n.taskList());
    }
}
```



Developer Experience

1. **Write i18n keys first:**

   type `i18n.notExistingLabel()` inside a gui component (e.g. TaskListView)

   → your IDE will mark the method as "not existing"

2. **Auto-Generate:**

   Use your IDE shortcut (`Ctrl + 1`) to create the missing method in the I18n interface

3. Continue programming in your View Component

4. **Fail-Fast Protection:**

   If you forget to add a mandatory `@TranslationXX` annotation, the application fails to start / reload



Improvement: Create a Language Interface

- A framework should not mandate a concrete implementation

  ⟹ I18nLanguage interface

```
public interface I18nLanguage {

    String  getCode();
    int     getIndex();
    boolean isMandatory();
    Locale  getLocale();
    String  getName();
    String  getName(Locale nameLocale);
    String  getName(I18nLanguage nameLanguage);

    Class<? extends Annotation> getAnnotationClass();
}
```



Improvement: **D***on't* **R***epeat* **Y***ourself*

- Do you want to repeat `switchLanguage` in every project ?

  → Probably NOT ⟹ use the frameworks `I18nBase` interface

```
public interface I18nBase {                // base interface in  easi18n  framework
    void switchLanguage(I18nLanguage language);
}
```

```
public interface I18n extends I18nBase {   // inherit 'switchLanguage()' in your project's I18n
}
```



Improvement: Divide and Conquer

- Do you want all labels and messages in one interface ?

  → Probably NOT ⟹ split it up into multiple interfaces

```
public interface I18nMsg {
    @TranslationDE("Es ist ein Fehler aufgetreten")
    @TranslationEN("An error occurred")
    String someFailureMessage();
}
```

```
public interface I18nLbl {
    @TranslationDE("Speichern")
    @TranslationEN("Save")
    String save();
}
```

```
public interface I18n extends I18nBase, I18nMsg, I18nLbl {
}
```



Improvement: External Translations

- Do you want all translations as annotations in the code ?

  → Probably NOT

```
// NOT USEFUL !!!

public interface I18n extends I18nBase {
    @TranslationDE("Sprache")
    @TranslationEN("Language")
    @TranslationES("Idioma")
    @TranslationIT("Lingua")
    @TranslationPT("Idioma")
    @TranslationPL("Język")
    @TranslationSE("Språk")
    @TranslationNO("Språk")
    @TranslationFI("Kieli")
    @TranslationCZ("Jazyk")
    @TranslationSK("Jazyk")
    @TranslationHU("Nyelv")
    @TranslationRO("Limbă")
    @TranslationGR("Γλώσσα")
    void save();
}
```



Improvement: External Translations

- Integrate an external Translation Source
  - `I18nBundle` has an `Optional<CustomTranslationService>`
  - method based `translationKey` is converted to `TRANSLATION_KEY`

```
public interface CustomTranslationService {
    String getTranslation(I18nLanguage language, String snakeCaseKey, Object[] args);  // e.g. SOME_FAILURE_MESSAGE
}
```

```
public class I18nBundle {
    final CustomTranslationService translationService;

    I18nBundle(..., Optional<CustomTranslationService> optionalTranslationService) { ... }

    String getTranslation(I18nLanguage language, String methodName, Object[] args) {  // methodName == translationKey
        // return  custom translation  if applicable  otherwise  return internal translation
    }
    String getCustomTranslation(I18nLanguage language, String methodName, Object[] args) {
        // convert "methodName" to "METHOD_NAME"  return  getTranslation from translationService
    }
```



More Improvements

- Springboot Starter Project with @AutoConfiguration
- Reusable Vaadin Components
  - AppLanguageWrapper
  - CustomTranslationManager: Manage you external translations
    - Multiple Sources (Database / REST Service / File)
    - Overview over all translations
    - See which translations are missing in a certain language
    - Check if translations are complete for one or more languages

⟹ Be excited about **easi18n**



Summary of Key Advantages:

- **Zero Runtime Failures:** Dead translation keys or wrong arguments are structurally impossible.
- **You deliver your application with a basic set of languages to the customer**
  - Since the setup is easy, you start new projects I18N ready right from the beginning
- **You know all possible translation keys**
  - Therefore you can check for completeness of translations even with external translation sources
- **IDE Support Out-Of-The-Box:** you just write normal interface methods
- **Ultra Lightweight:** Zero boilerplate code generation at compile time or plugin setups.
- **Immunity against Upgrades:** `java.lang.reflect.Proxy` has belonged to Java Core since JDK 1.3
  (⟹ no `javax.*` to `jakarta.*` mess expected)



THANK YOU

- Source Code and Slides available at:
  https://github.com/rschobbert/vaadin-create-2026-type-safe-i18n

- Questions || Comments || Suggestions ?
- Want to Talk ?
- Need Help ?

- Contact Me:
  ruediger.schobbert@emesit.de
  emesit GmbH & Co. KG

